

TECHNICAL DOCUMENTATION



Complete Project Techno

Python · Flask

Web Speech API

InterviewIQ is an AI-powered interview simulation platform that generates adaptive interview questions, evaluates candidate responses in real time, and produces comprehensive hiring reports — all powered by Google Gemini 2.0 Flash and built on a lightweight Python/Flask stack.



Backend

Server-side framework, authentication & security

TECHNOLOGY	VERSION	PURPOSE
Python	3.x	Core programming language
Flask	≥ 3.0.0	Web framework — routes, views, REST API endpoints
Flask-Login	≥ 0.6.3	User session management & authentication flow
Werkzeug	bundled	Password hashing (PBKDF2-SHA256)
Cryptography (Fernet)	≥ 41.0.0	Symmetric encryption for stored Gemini API keys



AI / Large Language Model

Question generation, answer evaluation & report compilation

TECHNOLOGY	VERSION	PURPOSE
Google Gemini 2.0 Flash	—	Core LLM — generates questions, evaluates answers, compiles reports
google-genai Primary	≥ 1.0.0	New Google GenAI SDK (preferred SDK)
google-generativeai Fallback	≥ 0.3.0	Legacy SDK used when new SDK is unavailable



Database

Local relational data storage

TECHNOLOGY	DETAILS	PURPOSE
SQLite3	Built-in (no install)	Lightweight relational database via <code>schema.sql</code>
users	Table	Stores credentials, encrypted API keys, account metadata
interviews	Table	Session config, scores, report fields, status & duration
questions	Table	Per-session generated questions with difficulty levels
answers	Table	Candidate responses with 6-dimension evaluation scores



Frontend

Templates, styling & interactive logic

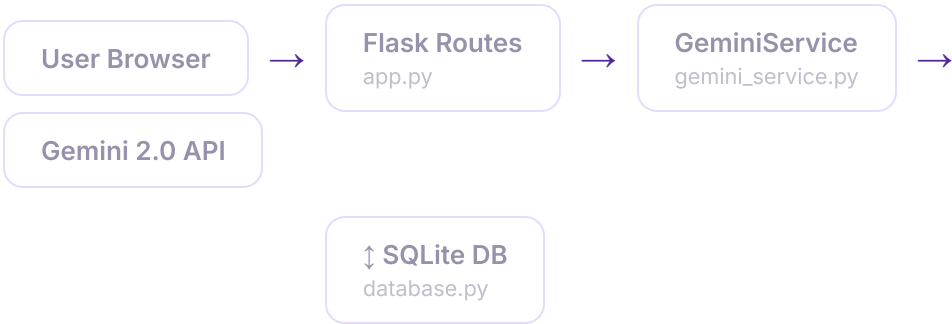
TECHNOLOGY	DETAILS	PURPOSE
HTML / Jinja2	12 templates	Landing, dashboard, interview, feedback, history, profile, settings...
Vanilla CSS	/static/css/	Full custom styling — no frameworks
Vanilla JavaScript	/static/js/	Interview session control, API calls, UI interactions
Web Speech API	Browser native	Voice input (STT) & voice output (TTS) — zero dependencies

TECHNOLOGY	DETAILS	PURPOSE
Chart.js	CDN	Score trend charts on the dashboard



Architecture

Request flow & design pattern



MVC Pattern — Routes in app.py, data layer in database.py, AI logic isolated in services/

Adaptive Difficulty — Question difficulty auto-adjusts based on live answer scores (≥80 → harder, <55 → easier)

Offline Fallback — Built-in mock question banks & evaluations work without any Gemini API key

SDK Dual Support — Automatically selects new google-genai SDK; falls back to google-generativeai gracefully